INSTITUTO FEDERAL DE MINAS GERAIS

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

CARLOS HENRIQUE LIMA

RELATÓRIO HERANÇA E POLIMORFISMO

SÃO JOÃO EVANGELISTA

2025

Introdução

Este relatório tem como objetivo descrever e analisar a atividade prática desenvolvida no componente curricular Seminário **Herança e Polimorfismo** na linguagem Java. A atividade buscou consolidar conceitos fundamentais da Programação Orientada a Objetos (POO), enfatizando estratégias para a construção de código reutilizável, escalável e de fácil manutenção.

Durante a aula, foram apresentados problemas comuns enfrentados em sistemas de grande porte, como a duplicidade de código, bem como soluções baseadas nos pilares da POO. Por meio de exemplos teóricos e atividades práticas, pudemos compreender como a herança e o polimorfismo contribuem para a organização, flexibilidade e extensibilidade de sistemas orientados a objetos.

Desenvolvimento

Inicialmente, discutiu-se o problema da **duplicidade de código**, comum em aplicações que não utilizam adequadamente os conceitos de herança. Utilizando como exemplo um sistema de Recursos Humanos, observou-se que diferentes classes (Gerente, Diretor, Secretária e Engenheiro) compartilham diversos atributos em comum, como nome, CPF e salário. Sem herança, esses atributos precisariam ser repetidos em cada classe, aumentando a redundância, a dificuldade de manutenção e a probabilidade de erros.

A partir desse cenário, foi introduzido o conceito de **herança**, definido como o mecanismo que permite a criação de novas classes a partir de classes já existentes, reaproveitando atributos e métodos. Destacou-se a relação lógica conhecida como regra do **"é um" (is-a)**, essencial para a correta aplicação da herança. Exemplos como "Gerente é um Funcionário" e "Cachorro é um Animal" ajudaram a diferenciar herança de outros relacionamentos, como composição.

Em seguida, abordou-se a sintaxe da herança em Java, por meio da palavra-chave **extends**, bem como as limitações da linguagem, que não suporta herança múltipla de classes. Também foram discutidos os **modificadores de acesso** (private, protected e public), com ênfase no uso do **protected** como alternativa adequada para permitir o acesso das subclasses sem comprometer o encapsulamento.

Outro ponto relevante foi a utilização da palavra-chave "**super"**, empregada para acessar construtores e métodos da superclasse. Ressaltou-se que a chamada ao construtor da classe

mãe deve ser a primeira instrução no construtor da classe filha, garantindo a correta inicialização dos atributos herdados.

As atividades práticas reforçaram esses conceitos por meio de diferentes contextos, como sistemas de funcionários, personagens de RPG e sistemas escolares. Nessas atividades, os alunos implementaram hierarquias de classes, adicionando atributos e métodos específicos às subclasses, validando na prática o reaproveitamento de código proporcionado pela herança.

Logo após, foi explorado o conceito de **polimorfismo**, definido como a capacidade de um objeto assumir várias formas. Demonstrou-se que uma variável do tipo da classe mãe pode referenciar objetos de subclasses, permitindo o tratamento genérico de diferentes tipos. Esse conceito foi aprofundado com a **sobrescrita de métodos (override)**, na qual subclasses redefinem comportamentos herdados para atender a necessidades específicas.

Discutiu-se também o uso de **classes abstratas**, que servem como modelos base para herança, não podendo ser instanciadas diretamente. Esse recurso foi apresentado como solução para evitar a criação de objetos genéricos sem significado prático, além de garantir que subclasses implementem comportamentos obrigatórios.

Por fim, foram apresentados exemplos práticos, como o cálculo de bonificações em um sistema corporativo e a ação de ataque em personagens de um jogo. Em todos os casos, o mesmo método era chamado, mas o comportamento executado variava conforme o tipo real do objeto em tempo de execução, caracterizando a **ligação tardia (late binding)**.

Conclusão

A atividade prática realizada no seminário possibilitou uma compreensão aprofundada dos conceitos de **herança e polimorfismo**, evidenciando sua importância no desenvolvimento de sistemas orientados a objetos robustos e bem estruturados. Por meio de exemplos concretos e exercícios práticos, foi possível observar como esses mecanismos reduzem a duplicidade de código, facilitam a manutenção e promovem a extensibilidade das aplicações.

Concluiu também que a correta aplicação da herança, aliada ao polimorfismo e à sobrescrita de métodos, contribui significativamente para a criação de softwares mais flexíveis e alinhados às boas práticas da POO. Dessa forma, os conhecimentos adquiridos na atividade prática representam uma base essencial para o desenvolvimento de sistemas profissionais em Java e em outras linguagens orientadas a objetos.